

 Universidad de los Andes Facultad de Ingeniería	Ingeniería de Sistemas y Computación ISIS-1611: Inteligencia artificial Semestre: 2026-19 Créditos: 3	
---	--	---

Taller 2: Simulador Adversario para la Copa Mundial de Fútbol

Colombia llega a la fase de eliminación directa del Mundial 2026 con la emoción de siempre, pero con una estrategia diferente: la Federación Colombiana de Fútbol se está transformando digitalmente y quiere tomar decisiones basadas en datos, no solo en la intuición del técnico. Por eso, el cuerpo técnico le ha encargado a usted diseñar un simulador de fútbol táctico. La idea es poder probar alineaciones, medir qué tan bien funcionan ciertas estrategias y repetir jugadas clave en la computadora antes de salir a la cancha de verdad.

En estas rondas finales, el rival también juega al máximo nivel. Cada partido en el simulador es un duelo directo donde Colombia busca ganar y el otro equipo hace todo por frenarla. Para resolver este problema táctico, tendrá que usar algoritmos de teoría de juegos incluyendo: Minimax para rivales que juegan perfecto y no se equivocan; Expectimax para manejar el azar (como el clima o las decisiones del árbitro); y Monte Carlo Tree Search (MCTS) para los momentos del partido donde las opciones se multiplican y calcular cada jugada se vuelve imposible.

Hoy en día, los partidos no se ganan solo corriendo, sino entendiendo cómo cada pase o presión cambia las probabilidades del juego. Los mejores equipos del mundo cuentan con equipos profesionales de analistas de datos y científicos computacionales que se encargan de reducir al máximo la incertidumbre. Su trabajo será modelar esa realidad: la impredecibilidad del balón, la dificultad de romper una defensa cerrada y la tensión de los últimos minutos. El cuerpo técnico busca un motor inteligente que le ayude a tomar la mejor decisión bajo presión.

Su tarea será construir este motor desde cero. Deberá definir cómo representar el estado del partido en código, programar una función de evaluación que premie el buen fútbol y comparar qué algoritmo funciona mejor según la situación. Saber qué herramienta usar en el momento adecuado será la clave para llevar a la "Sele" a la gran final en el MetLife Stadium el próximo 19 de julio.

Objetivos de aprendizaje

1. Abstraer el estado del juego para controlar la explosión combinatoria en búsqueda adversaria.
2. Implementar Minimax para un agente que compite contra un rival óptimo en un entorno discreto.
3. Implementar Expectimax para modelar rivales que actúan de forma probabilística.
4. Implementar MCTS con UCT como alternativa a la búsqueda en árbol explícita profunda.
5. Diseñar funciones de evaluación no triviales para estados intermedios de un juego adversario.
6. Analizar críticamente las propiedades de completitud, optimalidad, complejidad espacial y temporal de cada enfoque, y comparar experimentalmente su desempeño.

Escenario

El simulador opera sobre un campo rectangular con las siguientes reglas:

- Cada jugador se desplaza únicamente en 4 direcciones (arriba, abajo, izquierda, derecha).
- Cada jugador se mueve de forma independiente. Todos los jugadores se pueden mover en el mismo turno, y se desplazan exactamente al mismo tiempo (ambos equipos).
- Cada equipo cuenta con una portería propia que debe defender de su rival.
- El balón se encuentra en todo momento con uno de los jugadores del campo.
- El equipo con balón puede disparar al arco o hacer un pase al jugador más cercano.
- Se puede interceptar un disparo/pase para robar el balón si se encuentra en la trayectoria.
- Jugadores de distinto equipo pueden compartir posición de celda; hay robo de balón si un rival queda en la misma celda que el portador tras el movimiento.
- Se permiten escenarios con equipos asimétricos (distinto número de jugadores).
- El juego termina cuando cualquiera de los equipos realiza un gol a la portería contraria.
- Se declara empate si ninguno de los equipos realiza un gol en los turnos designados.

El simulador aplica las reglas de juego para determinar las nuevas posiciones de cada jugador, del balón y si hubo intercepción del balón.

Ejecución

El programa se puede ejecutar de forma general con el comando:

```
python src/main.py -a AGENTE -s ESCENARIO [OPCIONES]
```

Las opciones disponibles se describen a continuación:

Opción	Descripción	Ejemplo
-a	Agente tomador de decisión a utilizar.	-a MinimaxAgent
-s	Escenario YAML predefinido a cargar.	-s 1v1_defense
-d	Profundidad de búsqueda en el árbol de juego.	-d 3
-p	Probabilidad de que el rival ejecute una acción aleatoria (un rival greedy es $p = 0$ para $0 \leq p \leq 1$)	-p 0.2
-i	Número de iteraciones para Agente MCTS.	-i 300
-r	Semilla aleatoria para reproducibilidad.	-r 42
--help	Muestra la ayuda con todas las opciones disponibles.	--help

Puede verificar que el proyecto se ejecuta correctamente con el siguiente ejemplo de referencia:

```
python src/main.py -a RandomAgent -s 1v1_anticipation_block
```

Punto 1: Representación de estado y evaluación heurística (20%)

En un partido de eliminación directa no siempre se llega al gol en tres pases: hay presión, líneas tapadas, balón en zona incómoda y poco tiempo para pensar. El cuerpo técnico no puede simular

todos los partidos hasta el final, pero sí necesita que el simulador “entienda” si una jugada va bien encaminada: ¿estamos cerca del arco?, ¿el pase queda despejado?, ¿el rival nos está ahogando? Esa intuición táctica es justo lo que debe codificar en su función de evaluación: un puntaje que le diga a sus algoritmos qué tan prometedor es un estado cuando la búsqueda todavía no alcanza a ver el gol. Si la heurística es mala, el agente juega tímido o se lanza a ciegas; si es buena, Colombia empieza a tomar decisiones que se sienten de verdad en la cancha.

- a) Implemente en `src/algorithms/base/evaluation.py` la función `evaluation_function(state)`, que evalúa un estado no terminal del juego asignando un puntaje real desde la perspectiva de Colombia (valores más altos son mejores). En su informe justifique qué características del estado usa (distancia a portería, cobertura de líneas de tiro, presión sobre el portador, avance del balón, etc.), qué pesos asignó y por qué.

Pruebe su evaluación en distintos escenarios propios, compare cualitativamente el comportamiento de sus agentes y valide sus hipótesis. Ese proceso de experimentación puede enriquecer mucho su análisis. No se preocupe si su función no obtiene buenos resultados en todos los escenarios, lo importante es que pueda analizar por qué se desempeña mejor o peor en cada caso.

Punto 2: Rival Óptimo (30%)

Es el minuto 89. El tiempo se agota, el rival comete ni un solo error, y usted tiene que elegir la jugada que mejor se anticipe a lo peor que el otro puede hacer. Ese es el escenario de al que se enfrenta: un juego de suma cero donde Colombia intenta maximizar, mientras que el rival intenta minimizar, y ambos usan la misma función de evaluación para medir quién va ganando el pulso del partido.

- a) Implemente en `src/algorithms/minimax/search.py` la función de búsqueda con límite de profundidad. Cada ply corresponde a un timestep simultáneo del partido. Pruebe en escenarios con profundidad de 1 (y 2 donde el tiempo lo permita).
- b) Implemente en `src/algorithms/alphabeta/search.py` la función de búsqueda, con la misma estructura de ply que Minimax y poda por cotas. Verifique que AlphaBetaAgent elige exactamente las mismas acciones que MinimaxAgent explorando menos nodos.

Punto 3: Rival Estocástico (15%)

En la cancha real el rival no siempre es una máquina perfecta: a veces presiona con criterio, a veces se equivoca, a veces arriesga un movimiento que no era el óptimo. Usted modelará este comportamiento como una probabilidad p de ejecutar una acción al azar y una probabilidad $1 - p$ de ejecutar la acción greedy que minimiza la utilidad de Colombia. Así el simulador se prepara no solo para el rival genio, sino también para el partido impredecible.

- a) Implemente en `src/algorithms/expectimax/search.py` la función de búsqueda con valor esperado en el nodo rival. Experimente con varios valores de p .

Punto 4: Monte Carlo Tree Search (25%)

Cuando entran cinco jugadores por bando, el número de acciones hace que un árbol Minimax profundo sea impracticable incluso con Alpha-Beta. En lugar de expandir todo el árbol, se invierte

iteraciones en simular partidas parciales y aprender qué jugadas funcionan. Es la herramienta para escenarios grandes, donde se tienen demasiadas combinaciones para pensarlas exhaustivamente.

- a) Implemente UCT score en `src/algorithms/mcts/uct.py`.
- b) Implemente MCTS en `src/algorithms/mcts/search.py`.
- c) Compare distintos valores de iteraciones en escenarios y analice el balance entre calidad de juego y tiempo por turno.

Punto 5: Reflexión sobre el trabajo realizado (10%)

El cuerpo técnico de la Selección Colombia desea mejorar el simulador táctico para incrementar las probabilidades de éxito en futuros partidos. Con este fin, le ha solicitado un análisis a partir de su experiencia diseñando funciones de evaluación y algoritmos de búsqueda adversaria. Conteste las siguientes preguntas en un documento PDF:

- a) (6%) Respecto a la búsqueda en árbol (Minimax, Alpha-Beta, Expectimax), responda:
 - Complejidad en tiempo y espacio (notación O) en función de la profundidad d y el factor de ramificación b (acciones joint por equipo).
 - ¿Por qué Minimax puede ser pesimista en entornos con rivales no óptimos?
 - Describa y justifique su `evaluation_function`: ¿qué características del estado usa, qué pesos asignó y por qué?
 - Compare las tasas de victoria, derrota y empate de los agentes en diversos escenarios.
 - ¿Cuántos nodos adicionales poda Alpha-Beta en los mejores y peores casos?
- b) (6%) Respecto a MCTS, responda:
 - Complejidad en tiempo y espacio en función del número de iteraciones N y la ramificación del estado.
 - ¿En qué escenarios supera o no a Minimax y Expectimax? Justifique con experimentos.
 - Relación entre número de iteraciones y calidad de juego / tiempo por turno.

Se espera que en su análisis compare cualitativamente el desempeño de los distintos algoritmos en los escenarios que encuentre de interés, así como que compare cuantitativamente métricas de complejidad computacional y tiempo de ejecución.

- c) (3%) Escriba una reflexión concisa sobre la co-construcción de la solución final con apoyo de la IA, indicando, por ejemplo, qué aprendizajes obtuvo al ver las correcciones sugeridas, la dificultad para crear el *prompt* adecuado, si la solución propuesta fue fácil de entender (¿usted la podría explicar en una sustentación?). Si no hizo ninguna iteración con IA, indique por qué no le pareció necesario. Este punto busca que usted se tome el tiempo de reflexionar sobre el uso de la IA en tareas de programación.

Política de Uso de IA Generativa y presentación en el trabajo a entregar

Para este taller se espera que usted desarrolle una primera versión de la solución de forma completamente autónoma, usando su propio criterio, conocimiento y creatividad antes de recurrir a herramientas de IA. La IA puede emplearse después para mejoras puntuales, refactorización, comentarios de calidad o apoyo en la corrección de errores, pero nunca como sustituto del esfuerzo personal ni como generador principal del código. Use estas herramientas con criterio profesional,

asumiendo responsabilidad sobre su proceso de aprendizaje y entendiendo que lo que más valor tiene aquí es el camino que usted recorre para llegar a la solución, no solo el resultado final.

Todos los prompts y las versiones de código generadas con apoyo de IA deben quedar registrados dentro de los archivos del proyecto. Si utiliza IA para refactorizar u optimizar su solución, incluya en el archivo: (1) la versión inicial del código, (2) los prompts que utilizó para refinarla y (3) la versión final del código. Al finalizar, deje como código activo únicamente la versión final y conserve la versión inicial y todos los prompts en forma de comentarios dentro del mismo archivo.

Entrega del trabajo

- Guarde todo su trabajo (incluyendo el documento PDF) en una carpeta comprimida (.zip).
- Suba el trabajo al espacio correspondiente en Bloque Neón a más tardar la fecha y hora indicadas (un solo miembro del grupo debe subir el trabajo).
- Recuerde incluir los nombres y códigos de todos los integrantes de su grupo.

Nota: Al enviar su solución, usted declara que el código entregado en la primera versión de cada punto es de su autoría y que las versiones que utilizan IA fueron la respuesta a los prompts incluidos, que comprende el funcionamiento en todas las versiones y que acepta que este material pueda ser revisado, ejecutado y evaluado por el equipo docente para efectos académicos y de verificación de originalidad.